

Bash: Test Commands, Functions & Arrays

A comprehensive guide for developers and sysadmins covering shell test constructs, function patterns, and advanced string and array manipulation in modern Bash scripting.

BASH SCRIPTING SERIES

Test Commands & Logic

1

Test Constructs

Understand `test`, `[]`, and `[[ ]]`

2

File & String Tests

Operators for files, strings, and numeric comparisons

3

Logic & Combining

AND, OR, negation, and grouping conditions

4

Practical Patterns

Real-world validation and debugging strategies

Introduction to Test Commands

Bash provides three ways to evaluate conditions: the built-in `test` command, the POSIX-compatible `[]` brackets, and the modern Bash-specific `[[ ]]` construct. All three evaluate an expression and return an exit code — `$?=0` means **true**, and `$?=1` means **false**. Understanding this distinction is foundational: Bash's `if`, `while`, and `until` constructs act on these return codes, not on output text.

`test`

Traditional POSIX command. Works in all shells. Verbose syntax.

`[]`

Syntactic sugar for `test`. Requires spaces around brackets.

`[[ ]]`

Bash built-in. Supports regex, pattern matching, and safer quoting.

Test Command Syntax Comparison

While all three constructs serve similar purposes, they differ significantly in power and safety. The `[[ ]]` double bracket is the recommended choice for modern Bash scripts — it avoids word-splitting issues, supports pattern matching with `==`, and enables regular expression matching via `=~`.

Construct	Example	Key Behavior
<code>test</code>	<code>test -f file.txt</code>	POSIX portable, no special features
<code>[]</code>	<code>[-f file.txt]</code>	Same as test; quoting variables required
<code>[[]]</code>	<code>[[-f file.txt]]</code>	Bash only; pattern & regex matching, safer
<code>(())</code>	<code>((count > 5))</code>	Arithmetic context; use for numeric tests

📌 Always prefer `[[ ]]` in Bash scripts unless you need POSIX portability across shells like `sh` or `dash`.

File Testing Operators

File tests are among the most commonly used conditionals in shell scripts. They allow you to guard operations — like reading, writing, or executing — by verifying file properties before proceeding. Grouping operators by purpose makes them easier to recall.

Existence & Type

- `-e` — exists (any type)
- `-f` — regular file
- `-d` — directory
- `-L` — symbolic link
- `-b` — block device
- `-c` — character device
- `-p` — named pipe
- `-S` — socket

Permissions & Size

- `-r` — readable by current user
- `-w` — writable by current user
- `-x` — executable by current user
- `-s` — non-zero size
- `-t` — file descriptor is a terminal
- `-u` — SUID bit set
- `-g` — SGID bit set

String Testing Operations

String tests let you validate user input, check environment variables, and branch logic based on content. The `[[ ]]` construct unlocks two powerful capabilities unavailable in `[ ]`: glob-style **pattern matching** with `==` and full **regex matching** with `=~` . Captured regex groups are stored in the `BASH_REMATCH` array.

Empty / Non-Empty

`-z "$var"` — true if string is empty (zero length)

`-n "$var"` — true if string is non-empty

Equality

`==` — string equality (supports globs in `[[ ]]`)

`!=` — string inequality

Pattern Matching

`[[ "$f" == *.log ]]` — matches any `.log` file. Right-hand side is unquoted for glob expansion.

Regex Matching

`[[ "$s" =~ ^[0-9]+$ ]]` — true if string is all digits. Match groups available in `BASH_REMATCH`.

Numeric Comparisons

Bash handles numeric comparisons differently from strings. Use dedicated integer operators in `[]` or `[[ ]]`, or use the arithmetic context `(( ))` for a cleaner, C-style syntax. Mixing numeric operators with string values is a common source of bugs.

Integer Operators

<code>-eq</code>	Equal to
<code>-ne</code>	Not equal to
<code>-lt</code>	Less than
<code>-le</code>	Less than or equal
<code>-gt</code>	Greater than
<code>-ge</code>	Greater than or equal

Arithmetic Context `(( ))`

Use `(( ))` for natural numeric syntax:

```
(( count > 10 ))  
(( x == y * 2 ))  
(( ++i ))
```

Returns 0 (true) if expression is non-zero. Avoids quoting concerns and supports standard math operators.

❏ Never use `==` for numeric comparison in `[]` — it performs string comparison. Always use `-eq` or `(( ))`.

Combining Test Conditions

Complex conditions require logical operators to combine multiple tests. Bash supports two styles: shell-level operators (`&&`, `||`) which are universally recommended, and legacy test-internal operators (`-a`, `-o`) which should be avoided due to parsing ambiguity.



Logical AND

Prefer `&&` between two `[[ ]]` blocks. Avoid `-a` inside brackets — it's deprecated and ambiguous with multi-argument expressions.



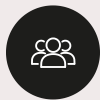
Logical OR

Use `| |` at the shell level. Avoid `-o` inside `[ ]` for the same reasons as `-a`.



Negation

The `!` operator inverts the result of a test. Works both inside `[[ ! -f file ]]` and outside `! command`.



Grouping

Inside `[[ ]]`, use `( )` directly. In `[ ]`, escape as `\( \)` — or better yet, switch to `[[ ]]`.

Practical Examples: File & String Validation

Real-world scripts rely heavily on conditional tests to ensure safety and correctness. Below are common patterns every Bash scripter should have in their toolkit.

1

Check if a command exists

`command -v git &>/dev/null && echo "git found"` — safer than `which`, works across distros.

2

Validate file before processing

`[[ -f "$file" && -r "$file" ]]` — confirms it's a readable regular file before parsing or sourcing.

3

Input validation with regex

`[[ "$input" =~ ^[a-zA-Z0-9_]+$ ]]` — ensures input contains only safe alphanumeric characters.

4

Guard against empty arguments

`[[ -z "$1" ]]` && { echo "Usage: \$0 <arg>"; exit 1; } — fail fast with a clear error message.

Functions & Return Values

Functions are the building blocks of maintainable Bash scripts. This section covers declaration syntax, parameter passing, return strategies, variable scope, and advanced techniques for writing clean, reusable shell functions.

Function Definition and Syntax

Bash supports two equivalent syntaxes for declaring functions. The `function` keyword form is more readable; the compact parenthesis form is POSIX-portable. Functions must be defined before they are called — place them near the top of your script or in a dedicated library file sourced at startup.

Two Declaration Styles

Style 1: function keyword

```
function greet() {  
    echo "Hello, $1"  
}
```

Style 2: POSIX-compatible

```
greet() {  
    echo "Hello, $1"  
}
```

Key Rules

- Names follow variable naming rules: letters, digits, underscores
- No spaces around `=` in assignments inside functions
- Functions share the script's global scope by default
- Use `local` to restrict variable visibility
- Call a function by name — no parentheses needed at call site

Function Parameters and Arguments

Functions receive arguments via positional parameters. Unlike most languages, Bash does not declare formal parameters — arguments are accessed as `$1`, `$2`, etc. For ten or more arguments, use `${10}` with braces. The special variables `$@` and `$*` differ crucially when quoted.

1

Positional Params

`$1–$9` and `${10}+` hold individual arguments passed at call time.

2

`$#` — Arg Count

Returns the number of arguments. Use it to validate the caller passed the required inputs.

3

`"$@"` — All Args

Expands each argument as a separate quoted word. The safe default for passing arguments forward.

4

`"$*"` — Joined Args

All arguments joined by the first character of `IFS`. Useful for building delimited strings.

Return Values: The `return` Statement

Bash functions communicate success or failure via **exit codes**, not values. The `return` statement accepts integers from 0–255. By convention, 0 signals success and any non-zero value signals failure. Use `$?` immediately after a function call to capture its code — any subsequent command will overwrite it.

return vs exit

- `return N` — exits the function, returns code to caller
- `exit N` — terminates the entire script process
- Never use `exit` inside a library function
- Omitting `return` returns exit code of last command run

Capturing the Code

```
validate_input "$arg"
status=$?
if (( status != 0 )); then
    echo "Error: code $status"
    exit 1
fi
```

Returning Strings from Functions

Because `return` only carries numeric codes, functions output string data via `echo` or `printf` to STDOUT. The caller captures this using **command substitution**. This is the standard Bash idiom for returning computed values, file contents, or processed strings.

1 Output via `echo`

The function writes to STDOUT: `echo "$result"`. The caller captures with `val=$(function_name)`. Clean, readable, and universally supported.

2 Capturing STDOUT vs STDERR

Command substitution captures only STDOUT. Redirect errors to STDERR explicitly: `echo "Error" >&2`. This keeps the return channel clean and separates logs from data.

3 Multiple Values

Output space- or newline-delimited values from one `echo`, then read into an array: `read -ra parts <<< "$(func)"` for safe multi-value returns.

Local Variables & Advanced Function Techniques

Without the `local` keyword, all variables in a function are global and can silently overwrite variables in the calling scope. Declaring locals is one of the highest-impact habits for writing bug-free Bash functions.

Variable Scope

```
process_file() {  
  local filename="$1"  
  local result  
  result=$(wc -l < "$filename")  
  echo "$result"  
}
```

`filename` and `result` are invisible outside the function — no leakage into global scope.

Advanced Techniques

- **Named return variables:** pass a variable name as an argument and use `printf -v "$1" "%s" "$value"` to assign into the caller's scope
- **Array returns:** declare an array in caller, pass its name, populate with `eval` or `nameref` (`declare -n`)
- **Error handling:** always validate `$?` or use `set -e` with traps to catch unexpected failures

SECTION 3 OF 3

String Manipulation & Arrays

Bash's parameter expansion syntax provides a powerful, zero-dependency toolkit for slicing, transforming, and manipulating strings and arrays entirely within the shell — no `awk`, `sed`, or external tools required.

Advanced String Expansion

Bash parameter expansion covers a wide range of string operations natively. Mastering these forms reduces subprocess spawning and speeds up scripts dramatically, especially in tight loops over large datasets.

Substring Extraction

`${string:pos:len}`

Extract `len` characters starting at `pos` (0-indexed). Negative positions count from the end: `${str: -4}`.

String Length

`${#string}`

Returns the number of characters. Works on variables, array elements, and special params like `${#@}`.

Default Values

`${var:-default}` — use default if unset or empty

`${var:=default}` — assign default if unset or empty

Case Conversion

`${var,,}` — convert to lowercase

`${var^^}` — convert to uppercase

`${var~}` — toggle first character case

Pattern Removal and Replacement

Bash provides anchored pattern removal from both ends of a string, plus in-place replacement — all without spawning a subshell. The `#` and `%` operators use glob patterns, not regex. The double forms `##`, `%%` use **greedy** (longest) matching.

Syntax	Example	Result
<code>\${str#pattern}</code>	<code>\${path#*/}</code>	Remove shortest match from start
<code>\${str##pattern}</code>	<code>\${path##*/}</code>	Remove longest match from start (basename)
<code>\${str%pattern}</code>	<code>\${file%.txt}</code>	Remove shortest match from end
<code>\${str%%pattern}</code>	<code>\${url%%/*}</code>	Remove longest match from end (protocol)
<code>\${str/pat/rep}</code>	<code>\${s/foo/bar}</code>	Replace first occurrence
<code>\${str//pat/rep}</code>	<code>\${s// /_}</code>	Replace all occurrences (global)

Arrays: Declaration, Access & Operations

Bash supports both **indexed** (integer-keyed) and **associative** (string-keyed) arrays. Indexed arrays are zero-based by default but can be sparse. Associative arrays require explicit declaration with `declare -A`.

Declaration

```
# Indexed array
fruits=(apple banana cherry)

# Associative array
declare -A colors
colors[sky]="blue"
colors[grass]="green"

# Sparse array
arr[0]="first"
arr[5]="sixth"
```

Access & Metadata

- `${arr[2]}` — access by index
- `${arr[@]}` — all elements (quoted separately)
- `${arr[*]}` — all elements (joined by IFS)
- `${#arr[@]}` — number of elements
- `${!arr[@]}` — all indices/keys
- `${arr[@]:1:3}` — slice: 3 elements starting at index 1
- `arr+=(new)` — append element
- `unset arr[2]` — remove element at index 2

Advanced Array Operations & Practical Patterns

Arrays unlock powerful data-processing patterns in pure Bash. Combining array operations with loops, parameter expansion, and string manipulation enables you to parse structured data, build lookup tables, and process lists — all without external tools.



Search & Replace in Arrays

`${arr[@]/pattern/replacement}`
applies a substitution across all elements at once, returning a new expanded list — the original array is unchanged.



Sorting Arrays

Bash lacks a native sort. The idiom is: `readarray -t sorted < <(<printf '%s\n' "${arr[@]}" | sort)`
— pipelines output into a new array via process substitution.



Multi-Dimensional Arrays

Bash has no native 2D arrays. Common workarounds: encode as `"row_col"` keys in an associative array, or serialize rows as delimited strings in an indexed array and split at access time.

📌 Always iterate with `"${arr[@]}"` (quoted) to preserve elements containing spaces or special characters.